Actividad 3: Programación Orientada a Objetos

SANTIAGO ALVAREZ NARANJO

Santiago Alvarez Naranjo
Docente: Walter Hugo Arboleda Mazo | Universidad Nacional de Colombia Sede Medellín

UNIVERSIDAD NACIONAL DE COLOMBIA

Contenido

Punto 1. 2
Enunciado: Ejercicio 8.2 página 483 2
Ejercicios propuestos. 2
Código 2
Imagen interfaz de usuario. 4
Diagrama de clases 4
Punto 2. 5
Enunciado: Ejercicio 8.3 página 495 5
Ejercicios propuestos. 5
Código 5
Imagen interfaz de usuario. 9
Diagrama de clases 10
URL Github: 10

UNIVERSIDAD NACIONAL DE COLOMBIA

Punto 1.

Enunciado: Ejercicio 8.2 página 483

Se requiere desarrollar un programa con interfaz gráfica de usuario que genere una ventana donde se solicite el ingreso de cinco notas obtenidas por un estudiante.
El programa debe calcular y mostrar en la parte inferior de la ventana los siguientes datos:

- El promedio de notas ingresadas.
- La desviación estándar de las notas ingresadas.
- La mayor nota obtenida.
- La menor nota obtenida.

Ejercicios propuestos.

- Modificar el programa del ejercicio para generar mensajes de alerta cuando se ingresen datos que no sean numéricos.
- Es obligatorio ingresar las cinco notas, se genera un mensaje de alerta cuando una nota no ha sido ingresada.

Código

```
## Punto 1: Ejercicio 8.2 pagina 483
## Punto 1: Ejercicio 8.2 pagina 483
import tkinter as tk
import math

class CalculadorNotas:
    def __init__(self, ventana):
        self.ventana = ventana
        self.ventana.title("Calculadora de Notas")
        self.entries = []
        for i in range(5):
            frame = tk.Frame(self.ventana)
            frame.pack(pady=5)
            tk.Label(frame, text=f"Nota {i+1}:").pack(side=tk.LEFT)
            entry = tk.Entry(frame, width=10)
            entry.pack(side=tk.LEFT, padx=5)
            self.entries.append(entry)

        tk.Button(self.ventana, text="Calcular",
command=self.calcular).pack(pady=15)

        self.lbl_prom = tk.Label(self.ventana, text="Promedio: ")
        self.lbl_prom.pack()
        self.lbl_desv = tk.Label(self.ventana, text="Desviación estándar: ")
        self.lbl_desv.pack()
```

```
        self.lbl_max = tk.Label(self.ventana, text="Mayor nota: ")
        self.lbl_max.pack()
        self.lbl_min = tk.Label(self.ventana, text="Menor nota: ")
        self.lbl_min.pack()

    def calcular(self):
        notas = []
        for i, entry in enumerate(self.entries):
            val = entry.get().strip()
            if not val:
                messagebox.showerror("Error", f"Falta ingresar la nota
{i+1}.")
                return
            try:
                notas.append(float(val))
            except ValueError:
                messagebox.showerror("Error", f"La nota {i+1} debe ser un
número válido.")
                return

        prom = sum(notas) / 5
        var = sum((x - prom) ** 2 for x in notas) / 5
        desv = math.sqrt(var)
        max_n = max(notas)
        min_n = min(notas)

        self.lbl_prom.config(text=f"Promedio: {prom:.2f}")
        self.lbl_desv.config(text=f"Desviación estándar: {desv:.2f}")
        self.lbl_max.config(text=f"Mayor nota: {max_n:.2f}")
        self.lbl_min.config(text=f"Menor nota: {min_n:.2f}")

root = tk.Tk()
app = CalculadorNotas(root)
root.mainloop()
```

Imagen interfaz de usuario.

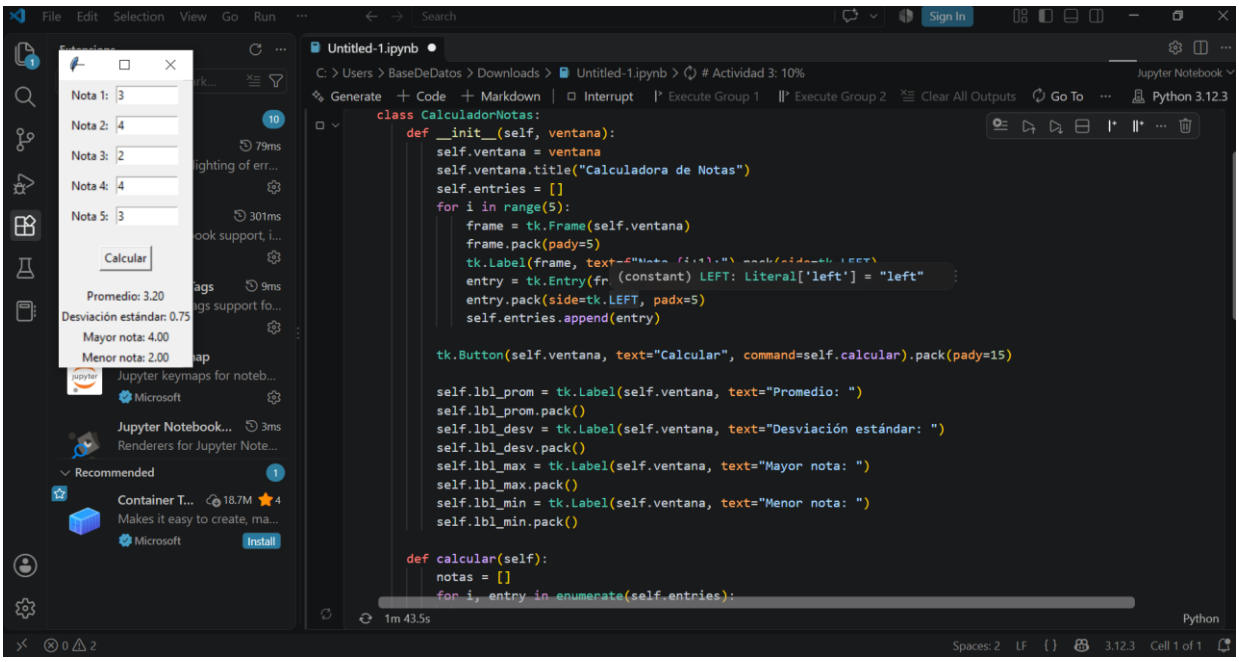
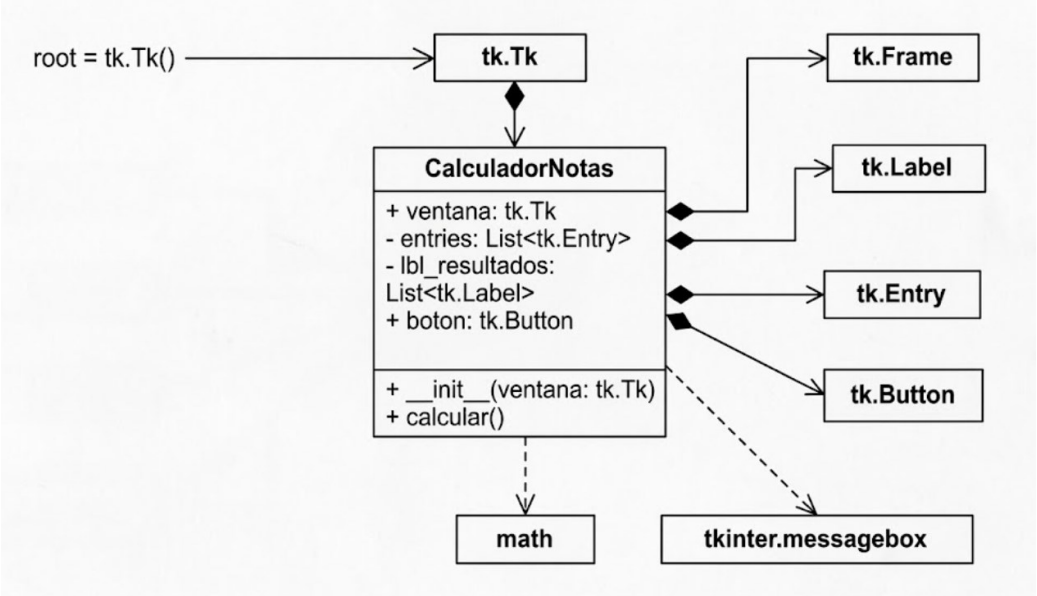


Diagrama de clases



Punto 2.

Enunciado: Ejercicio 8.3 página 495.

Se requiere desarrollar un programa con interfaz gráfica de usuario que permita calcular el volumen y superficie de varias figuras geométricas. Las figuras geométricas son el cilindro, la esfera y la pirámide.

- Para el cilindro se solicitan su radio y altura (en centímetros).
- Para la esfera, su radio (en centímetros).
- Para la pirámide, su base, altura y apotema (en centímetros).

Una vez ingresados estos datos, el programa calcula el volumen y superficie de cada figura. Para desarrollar el programa se debe crear una jerarquía de clases para las diferentes figuras geométricas requeridas.

Ejercicios propuestos.

- Agregar el cálculo del volumen y superficie de otras figuras geométricas como el cubo y el prisma.
- Agregar a cada figura geométrica correspondiente su imagen correspondiente.

Código

```
## Punto 2: Ejercicio 8.3 página 495
import tkinter as tk
from tkinter import messagebox, ttk
import math

class FiguraGeometrica:
    def calcular_volumen(self):
        pass
    def calcular_superficie(self):
        pass

class Cilindro(FiguraGeometrica):
    def __init__(self, radio, altura):
        self.radio = radio
        self.altura = altura
    def calcular_volumen(self):
        return math.pi * (self.radio ** 2) * self.altura
    def calcular_superficie(self):
        return 2 * math.pi * self.radio * (self.radio + self.altura)

class Esfera(FiguraGeometrica):
    def __init__(self, radio):
        self.radio = radio
    def calcular_volumen(self):
        return (4 / 3) * math.pi * (self.radio ** 3)
    def calcular_superficie(self):
```

```
        return 4 * math.pi * (self.radio ** 2)

class Piramide(FiguraGeometrica):
    def __init__(self, base, altura, apotema):
        self.base = base
        self.altura = altura
        self.apotema = apotema
    def calcular_volumen(self):
        return ((self.base ** 2) * self.altura) / 3
    def calcular_superficie(self):
        return (self.base ** 2) + 2 * self.base * self.apotema

class Cubo(FiguraGeometrica):
    def __init__(self, lado):
        self.lado = lado
    def calcular_volumen(self):
        return self.lado ** 3
    def calcular_superficie(self):
        return 6 * (self.lado ** 2)

class PrismaRectangular(FiguraGeometrica):
    def __init__(self, base, ancho, altura):
        self.base = base
        self.ancho = ancho
        self.altura = altura
    def calcular_volumen(self):
        return self.base * self.ancho * self.altura
    def calcular_superficie(self):
        return 2 * (self.base * self.ancho + self.base * self.altura + self.ancho * self.altura)

class CalculadorFiguras:
    def __init__(self, ventana):
        self.ventana = ventana
        self.ventana.title("Calculadora de Figuras Geométricas")
        self.ventana.geometry("380x420")
        tk.Label(self.ventana, text="Seleccione Figura:").pack(pady=5)
        self.combo_figura = ttk.Combobox(self.ventana, values=["Cilindro", "Esfera", "Pirámide",
"Cubo", "Prisma"], state="readonly")
        self.combo_figura.current(0)
        self.combo_figura.pack(pady=5)
        self.combo_figura.bind("<<ComboboxSelected>>", self.cambiar_figura)
        self.frame_inputs = tk.Frame(self.ventana)
        self.frame_inputs.pack(pady=10)
        self.canvas_imagen = tk.Canvas(self.ventana, width=120, height=120, bg="lightgrey")
        self.canvas_imagen.pack(pady=5)
```

```
    tk.Button(self.ventana, text="Calcular", command=self.calcular).pack(pady=10)

    self.lbl_volumen = tk.Label(self.ventana, text="Volumen: ")
    self.lbl_volumen.pack()
    self.lbl_superficie = tk.Label(self.ventana, text="Superficie: ")
    self.lbl_superficie.pack()

    self.entries = {}
    self.cambiar_figura(None)

def cambiar_figura(self, event):
    for widget in self.frame_inputs.winfo_children():
        widget.destroy()
    self.entries.clear()

    figura = self.combo_figura.get()
    campos = []
    if figura == "Cilindro":
        campos = ["Radio (cm)", "Altura (cm)"]
    elif figura == "Esfera":
        campos = ["Radio (cm)"]
    elif figura == "Pirámide":
        campos = ["Base (cm)", "Altura (cm)", "Apotema (cm)"]
    elif figura == "Cubo":
        campos = ["Lado (cm)"]
    elif figura == "Prisma":
        campos = ["Base (cm)", "Ancho (cm)", "Altura (cm)"]

    for campo in campos:
        frame = tk.Frame(self.frame_inputs)
        frame.pack(pady=2)
        tk.Label(frame, text=f"{campo}:", width=12, anchor="e").pack(side=tk.LEFT)
        entry = tk.Entry(frame, width=10)
        entry.pack(side=tk.LEFT, padx=5)
        self.entries[campo] = entry

    self.dibujar_imagen(figura)

def dibujar_imagen(self, figura):
    self.canvas_imagen.delete("all")
    if figura == "Cilindro":
        self.canvas_imagen.create_oval(30, 20, 90, 40, fill="#a0c4ff")
        self.canvas_imagen.create_rectangle(30, 30, 90, 90, fill="#a0c4ff", outline="")
        self.canvas_imagen.create_line(30, 30, 30, 90)
```

```
            self.canvas_imagen.create_line(90, 30, 90, 90)
            self.canvas_imagen.create_oval(30, 80, 90, 100, fill="#a0c4ff")
        elif figura == "Esfera":
            self.canvas_imagen.create_oval(20, 20, 100, 100, fill="#ffadad")
            self.canvas_imagen.create_oval(20, 50, 100, 70, outline="gray")
        elif figura == "Pirámide":
            self.canvas_imagen.create_polygon(60, 20, 50, 100, 100, 90, fill="#caffbf")
            self.canvas_imagen.create_polygon(50, 100, 20, 90, 60, 20, fill="#b5ead7")
        elif figura == "Cubo":
            self.canvas_imagen.create_rectangle(20, 40, 70, 90, fill="#fdffb6")
            self.canvas_imagen.create_polygon(20, 40, 40, 20, 90, 20, 70, 40, fill="#fdffb6",
outline="black")
            self.canvas_imagen.create_polygon(70, 40, 90, 20, 90, 70, 70, 90, fill="#fdffb6",
outline="black")
        elif figura == "Prisma":
            self.canvas_imagen.create_rectangle(20, 40, 80, 90, fill="#ffd6a5")
            self.canvas_imagen.create_polygon(20, 40, 40, 20, 100, 20, 80, 40, fill="#ffd6a5",
outline="black")
            self.canvas_imagen.create_polygon(80, 40, 100, 20, 100, 70, 80, 90, fill="#ffd6a5",
outline="black")

    def calcular(self):
        valores = {}
        for campo, entry in self.entries.items():
            val = entry.get().strip()
            if not val:
                messagebox.showerror("Error", f"Falta ingresar {campo}.")
                return
            try:
                valores[campo] = float(val)
            except ValueError:
                messagebox.showerror("Error", f"{campo} debe ser un número válido.")
                return

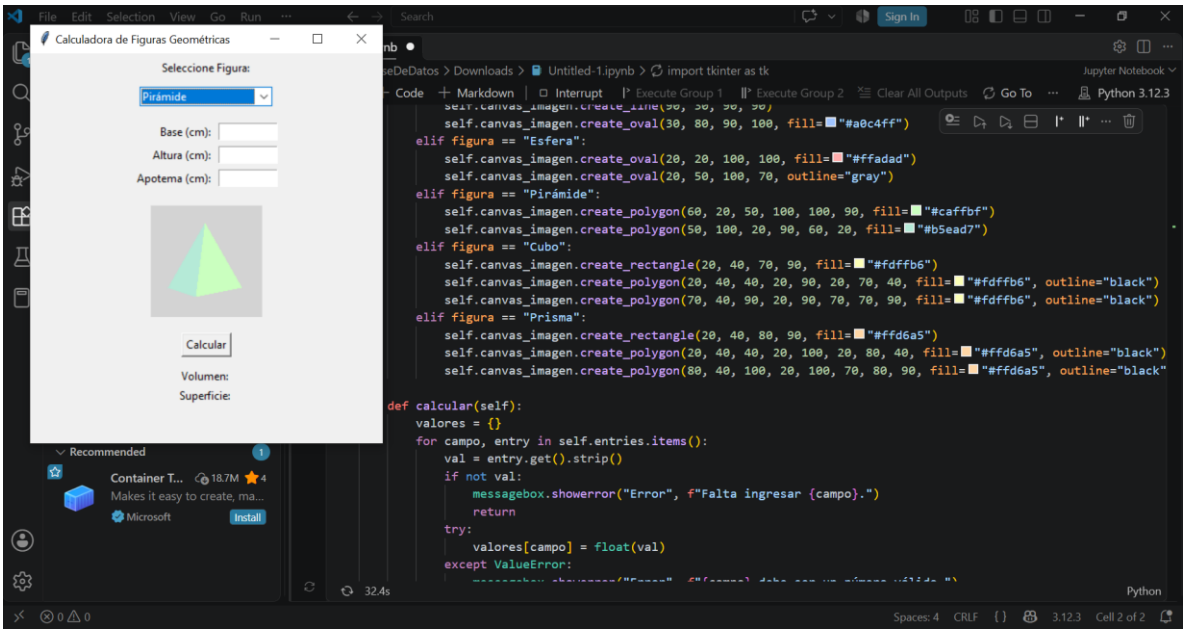
        figura_tipo = self.combo_figura.get()
        if figura_tipo == "Cilindro":
            obj = Cilindro(valores["Radio (cm)"], valores["Altura (cm)"])
        elif figura_tipo == "Esfera":
            obj = Esfera(valores["Radio (cm)"])
        elif figura_tipo == "Pirámide":
            obj = Piramide(valores["Base (cm)"], valores["Altura (cm)"], valores["Apotema (cm)"])
        elif figura_tipo == "Cubo":
            obj = Cubo(valores["Lado (cm)"])
        elif figura_tipo == "Prisma":
            obj = PrismaRectangular(valores["Base (cm)"], valores["Ancho (cm)"], valores["Altura (cm)"])
```

```
        vol = obj.calcular_volumen()
        sup = obj.calcular_superficie()

        self.lbl_volumen.config(text=f"Volumen: {vol:.2f} cm³")
        self.lbl_superficie.config(text=f"Superficie: {sup:.2f} cm²")

root = tk.Tk()
app = CalculadorFiguras(root)
root.mainloop()
```

Imagen interfaz de usuario.



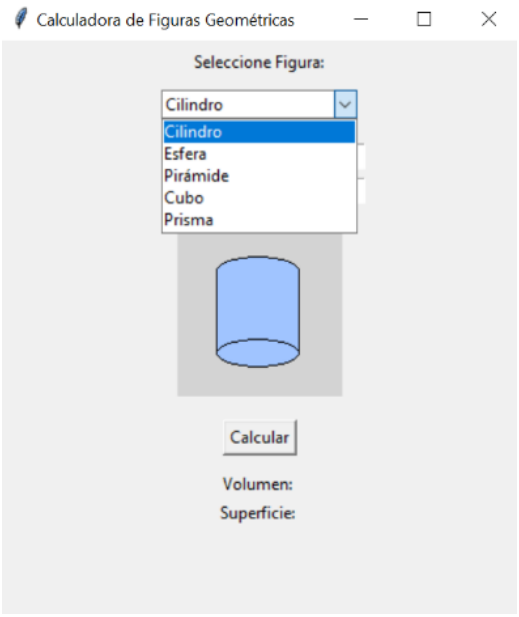
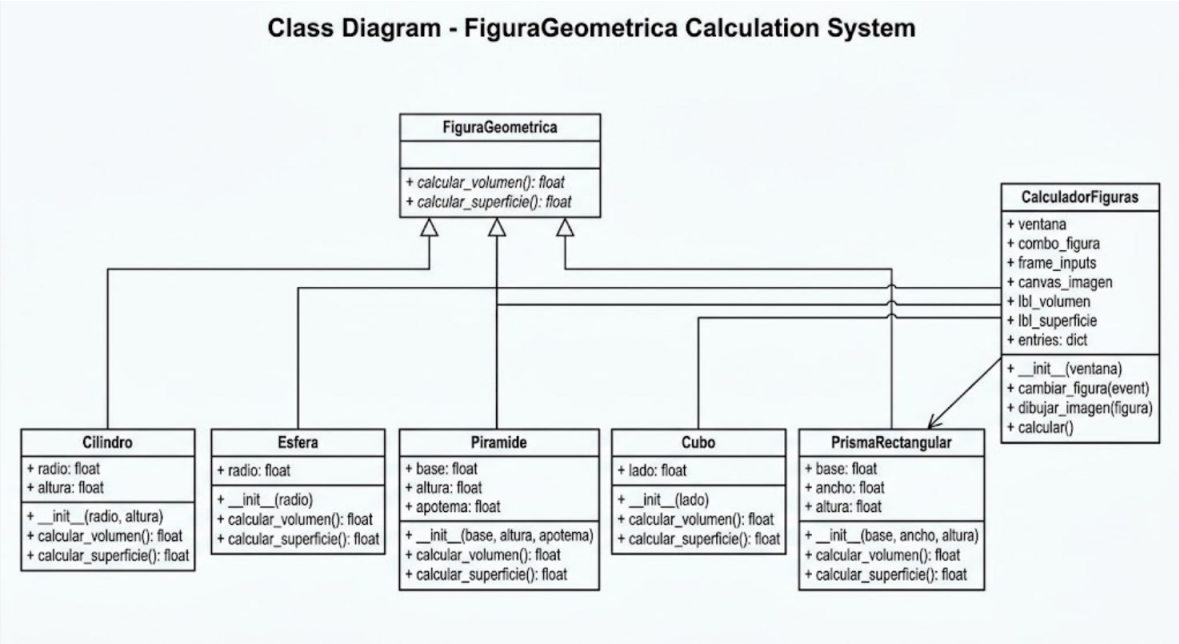


Diagrama de clases.



URL Github:

https://github.com/Santi0405/Actividades-POO